

Lesson 8 — What Each Notebook Shows

Notebook guide, lesson 8 — Technologies for Artificial Intelligence

Fabio Antonini — Università degli Studi dell'Aquila

20 November 2026 · three notebooks

Contents

Notebook 01 — 01_kmeans_from_scratch	1
k-means from scratch, and how many clusters there are	1
Notebook 02 — 02_hierarchical_and_dbscan	2
When k-means fails: hierarchical clustering and DBSCAN	2
Notebook 03 — 03_pca_and_anomaly_detection	4
Principal component analysis (PCA) from scratch, and the anomalies a 2-D plot cannot show	4

A short guide to this lesson's notebooks. For each one: what it is for, the concepts in the order they appear, the numbers worth noticing, and **the two-minute version**, a summary to read before running it or to come back to afterwards. Every number here comes from the committed output of the notebook it belongs to.

Notebook 01 — 01_kmeans_from_scratch

k-means from scratch, and how many clusters there are

What it is for. The first notebook of the course with **no y anywhere**. Aurora, an online retailer, wants to segment 2,000 customers that nobody has ever labelled.

The data. 2,000 customers, two features, **four true segments never shown to any model**.

The concepts, in order.

1. **Look at it first, unlabelled** — which is what the marketing team actually has. Four clouds are visible to the eye, and that is exactly the property k-means is built to exploit.
2. **The objective:** minimise the within-cluster sum of squares by alternating assignment and update. Each step can only lower J , so it always converges — **to a local minimum**.
3. **A bad start is a real risk, not a technicality.** Thirty naive starting points: best WCSS **374.1**, worst **1390.4**, a ratio of **3.72**× on identical data with only the initialisation different. k-means++ narrows it: best 374.1, worst 1088.3, mean **397.9** against 408.0. But count the

runs: **29 of 30** reach 374.1 either way — k-means++ made the bad case less bad, not rarer (kmeans_init_spread.png).

4. **Checked against scikit-learn:** from scratch 374.0919, scikit-learn 374.0919, difference **0.000000**.
5. **How many clusters?** WCSS and silhouette, and what each measures:

k	WCSS	silhouette
2	2128.5	0.472
3	1125.8	0.564
4	374.1	0.690
5	320.6	0.593
8	210.4	0.474

WCSS always falls, so only its *elbow* means anything; the silhouette peaks. **Both agree on 4 here, and say that the agreement is not guaranteed.**

6. **Checked against the withheld truth:** adjusted Rand index **0.9850**.
7. **The predictable mistake**, named explicitly: having watched k-means recover this almost perfectly, most students conclude clustering is generally reliable. The reasoning is sound *for data shaped like a handful of round blobs* — which notebook 2 then takes away.

The two-minute version. *Two thousand customers, no labels anywhere. k-means alternates two steps and always converges, but only to a local minimum: thirty naïve starts on the same data give within-cluster sums of squares from 374 to 1390, which is why k-means++ and ten restarts are the defaults. Choosing k uses two diagnostics that measure different things — WCSS falls forever so you look for its elbow, the silhouette peaks — and here both say four. Against the segments we withheld, the adjusted Rand index is 0.985. And the warning the next notebook collects: that worked because this data really is four round blobs.*

Notebook 02 — 02_hierarchical_and_dbscan

When k-means fails: hierarchical clustering and DBSCAN

What it is for. To take the previous notebook’s success away, and show that “clustering” is not one method with one set of assumptions.

The data. 1,500 web sessions, **12% generated as bots**, label withheld — one dense smudge sitting *inside* a larger looser cloud, both centred in the same place.

The concepts, in order.

1. **k-means, applied anyway:** ARI **−0.046**, which is a random split of the same sizes. **Not a tuning failure** — the objective itself is wrong for this shape.
2. **Hierarchical clustering**, with all four linkage rules cut at two clusters:

linkage	ARI	cluster sizes
ward	−0.062	1007 / 493
complete	−0.101	1253 / 247
average	−0.001	1499 / 1
single	−0.001	1499 / 1

Ward and complete fail for k-means’ reason; single and average chain everything into one cluster and leave a single point outside.

3. **DBSCAN: density, not shape.** With `eps=0.30`, `min_samples=10`: **ARI 0.9408**, 1,435 core points, 13 noise points. Every one of the 180 bot sessions in one cluster, 1,303 of 1,320 humans in the other.
4. **The internal metric points the wrong way:** silhouette **0.401** for k-means’ useless split against **−0.106** for DBSCAN’s correct one — two compact halves are rounder than a core inside a ring. Only the ARI sees which is right.
5. **And the noise points are the honest part:** all 13 are genuine humans whose week happened to look script-like. DBSCAN’s answer for them is neither “bot” nor “human” — it is “I am not going to say”, which no other method here can express.
6. **Does eps transfer?** Seven fresh weeks from the same generator with the same constant:

seed	ARI at <code>eps = 0.30</code>	clusters found
8002	0.941	2
3	−0.012	1
4	−0.009	1
5	0.955	2

The constant does not transfer. On two of seven weeks DBSCAN returns a single cluster and the result collapses — which is what “tuned on this week’s data” means in practice.

The two-minute version. *Same company, different problem: bot sessions hidden inside human ones, one dense cloud inside a looser one with the same centre. k-means scores an adjusted Rand index of −0.046, which is what a random split scores, and hierarchical clustering does no better — Ward and complete linkage fail for the same reason, single and average put 1,499 points in one cluster. DBSCAN, which asks about density rather than shape, gets 0.941 and puts all 180 bots in one cluster. Its 13 “noise” points are all genuine humans, and refusing to classify them is the honest answer. Then the caution: run it on seven other weeks with the same eps and two of them collapse to a single cluster. The parameter was read off this week’s data.*

Notebook 03 — 03_pca_and_anomaly_detection

Principal component analysis (PCA) from scratch, and the anomalies a 2-D plot cannot show

What it is for. Compression as a modelling decision, and then a use for it that a scatter plot cannot replace.

The data. 2,000 accounts, 8 behavioural columns, generated from **3 latent factors** (withheld) with **40 planted anomalies**.

The concepts, in order.

1. **Why compress at all:** the eight columns are not independent — spend, basket value and mobile sessions all partly reflect the same underlying tendency.
2. **PCA by eigendecomposition.** The directions of greatest variance, perpendicular to each other: PC1 **44.2%**, PC2 32.1%, PC3 17.2%, and then a collapse to 2.4% and below. Then the same idea on two columns, drawn (`pca_two_columns.png`): on spend and visits PC1 is the diagonal (variance 1.171), PC2 across it (0.830), and customer 13 reconstructed from PC1 alone leaves a perpendicular residual of squared length **6.37** — reconstruction error, in two dimensions.
3. **Three routes to the same answer.** Eigendecomposition of the covariance, $s_i^2/(m-1)$ from the SVD, and scikit-learn's `explained_variance_`, agreeing to 4.9×10^{-15} .
4. **How many components.** The first three explain **93.5%**; the fourth adds 2.4% — the scree plot recovers, from the data alone, the number the notebook opened by withholding.
5. **Anomaly detection by reconstruction error.** Keep the top components, map back, and measure the distance. Mean reconstruction error **4.676 for anomalies against 0.431 for genuine accounts**; flagging the top 2% catches **37 of 40** (92.5% precision and recall).
6. **With the honesty to repeat it.** Across eight batches the count caught runs **32 to 38**, median 36. Carry 37 as the memorable figure and the range as what it means.
7. **Why a 2-D plot is the wrong tool here.** Project to two components and the anomalies are **not** the outliers — mean distance from the genuine centroid **1.30 for anomalies against 2.20 for genuine accounts** in PCA space, and 22.16 against 30.17 under t-SNE. They sit *closer* to the middle in both, and it replicates at other seeds.
8. **The predictable mistake**, named: notebook 1 taught that a 2-D scatter is the fastest way to see structure, and the instinct to reach for one here is not unreasonable. The failure is treating “plot it and look” as a detector.

The two-minute version. *Eight columns that are not independent, generated from three latent factors we withhold. PCA finds the directions of greatest variance — three routes to the same eigenvalues, agreeing to fifteen decimal places — and the first three explain 93.5% of the variance, recovering the hidden number from the data alone. Then the use: reconstruct each account from those three components and measure the error. The anomalies reconstruct badly, 4.68 against 0.43, so the top 2% catches 37 of the 40 planted ones. And the last section is the one to keep: project the same data to two dimensions and the anomalies are not visible as outliers at all — on average they sit closer to the centre than genuine accounts do, in both PCA and t-SNE. Looking at a picture is not a detector.*